

Práctico AYED1 - 02-09 - Sala 2

Práctico 2 - Ejercicio 2d

Dar el tipo y derivar:

$\text{cuantos.p.xs} = \langle N \ i : 0 \leq i < \#xs : p.(xs[i]) \rangle$

Tipos de los parámetros:

- $xs : [A]$
- Acá vemos que p aparece en el término usado como una función (aplicado a algo, “ $p.$ ”)
- $p : A \rightarrow \text{Bool}$

Tipo de la función:

- $\text{cuantos} : (A \rightarrow \text{Bool}) \rightarrow [A] \rightarrow \text{Nat}$
- “ cuantos ” es una función que toma otra función como parámetro (alto orden).

Recordemos la definición del cuantificador de conteo:

$$\langle N \ i : R.i : T.i \rangle = \langle \sum \ i : R.i \wedge T.i : 1 \rangle$$

Ojo: no valen las reglas del digesto para cuantificadores generales (tiene su propio digesto, ver Anexo)

Derivación: Lo haremos por inducción en xs .

Caso base: $xs = []$.

cuantos.p.[]
 = { especificación }
 $\langle N i : 0 \leq i < \# [] : p.([]!i) \rangle$
 = { def # }
 $\langle N i : 0 \leq i < 0 : p.([]!i) \rangle$
 = { lógica }
 $\langle N i : \text{False} : p.([]!i) \rangle$
 = { rango vacío del conteo }
 0

Paso inductivo: $xs = y \blacktriangleright ys$

Hipótesis Inductiva (HI): $\text{cuantos.p.ys} = \langle N i : 0 \leq i < \#ys : p.(ys!i) \rangle$

cuantos.p.(y $\blacktriangleright$ ys)
 = { especificación }
 $\langle N i : 0 \leq i < \#(y \blacktriangleright ys) : p.((y \blacktriangleright ys)!i) \rangle$
 = { def # }
 $\langle N i : 0 \leq i < \#ys + 1 : p.((y \blacktriangleright ys)!i) \rangle$
 = { lógica, aritmetica, cualquier cosa q viene de otra materias }
 $\langle N i : i = 0 \vee 1 \leq i < \#ys + 1 : p.((y \blacktriangleright ys)!i) \rangle$
 = { partición de rango para conteo (con predicados disjuntos) }
 $\langle N i : i = 0 : p.((y \blacktriangleright ys)!i) \rangle + \langle N i : 1 \leq i < \#ys + 1 : p.((y \blacktriangleright ys)!i) \rangle$
 = { rango unitario del conteo }
 ~~$p.((y \blacktriangleright ys)!0) + \langle N i : 1 \leq i < \#ys + 1 : p.((y \blacktriangleright ys)!i) \rangle$~~ MAL!! ni siquiera tipa esto, suma booleano a un numero.
 $(p.((y \blacktriangleright ys)!0) \rightarrow 1$
 $\square \neg p.((y \blacktriangleright ys)!0) \rightarrow 0$
 $)$
 $+ \langle N i : 1 \leq i < \#ys + 1 : p.((y \blacktriangleright ys)!i) \rangle$
 = { def de ! }

```

( p.y → 1
  □ ¬ p.y → 0
)
+ ⟨ N i : 1 ≤ i < #ys + 1 : p.((y ► ys)!i) ⟩
= { cambio de variable i → i + 1 (f.i = i + 1, reusamos el mismo nombre de variable) }
( p.y → 1
  □ ¬ p.y → 0
)
+ ⟨ N i : 1 ≤ i + 1 < #ys + 1 : p.((y ► ys)!(i+1)) ⟩
= { resto 1 en la desigualdad }
( p.y → 1
  □ ¬ p.y → 0
)
+ ⟨ N i : 0 ≤ i < #ys : p.((y ► ys)!(i+1)) ⟩
= { def. de ! }
( p.y → 1
  □ ¬ p.y → 0
)
+ ⟨ N i : 0 ≤ i < #ys : p.(ys!i) ⟩
= { H.I. }
( p.y → 1
  □ ¬ p.y → 0
)
+ cuantos.p.ys
= { ya esta, pero hago un pasito más para dejar en análisis por casos afuera de todo }
( p.y → 1 + cuantos.p.ys
  □ ¬ p.y → 0 + cuantos.p.ys
)
= { neutro + }
( p.y → 1 + cuantos.p.ys
  □ ¬ p.y → cuantos.p.ys
)

```

Listo!! Resultado final:

$\text{cuantos} : (A \rightarrow \text{Bool}) \rightarrow [A] \rightarrow \text{Nat}$

$\text{cuantos.p.[]} \doteq 0$

$\text{cuantos.p.}(x \blacktriangleright xs) \doteq \begin{pmatrix} p.x \rightarrow 1 + \text{cuantos.p.}xs \\ \square \neg p.x \rightarrow \text{cuantos.p.}xs \\ \end{pmatrix}$

Ejemplo de uso de cuantos:

sea $f.n = \text{"n tiene un dígito que es un 5"}$

$\text{cuantos.f.}[23, 56, -5, -1, 7] = 2$

Práctico 2 - Ejercicio 2f

Dar el tipo y derivar:

$\text{busca.e.xs} = \langle \text{Min } i : 0 \leq i < \# xs \wedge xs!i = e : i \rangle$

¿Qué calcula busca? "La primer posición de xs en la que aparece e."

Ejemplo: $\text{busca.0.}[1, 0, 2, 0] = 1$

Ejemplo 2: $\text{busca.10.}[1, 0, 2, 0] = + \text{ infinito}$

Ejemplo 3: $\text{busca.}[32].[\text{ [0] }, [\text{ }], [\text{ 21, 32 }], [\text{ 32 }]] = 3$

Tipo de la función: $\text{busca} : A \rightarrow [A] \rightarrow \text{Nat}$

Derivación: Por inducción en xs.

Caso base: $xs = []$

$\text{busca.e.}[]$
= { especificación }
 $\langle \text{Min } i : 0 \leq i < \#[] \wedge []!i = e : i \rangle$
= { def. # }
 $\langle \text{Min } i : 0 \leq i < 0 \wedge []!i = e : i \rangle$
= { cursillo }
 $\langle \text{Min } i : \text{False} \wedge []!i = e : i \rangle$
= { intro a los algoritmos }
 $\langle \text{Min } i : \text{False} : i \rangle$
= { rango vacío }
+ infinito

(+ infinito es parte del lenguaje de programación del teo/prá así que ya estamos)

Paso inductivo: $xs = y \blacktriangleright ys$

$\text{busca.e.}(y \blacktriangleright ys)$
= { especificación }
 $\langle \text{Min } i : 0 \leq i < \#(y \blacktriangleright ys) \wedge (y \blacktriangleright ys)!i = e : i \rangle$
= { def. # }
 $\langle \text{Min } i : 0 \leq i < \#ys + 1 \wedge (y \blacktriangleright ys)!i = e : i \rangle$
= { lógica }
 $\langle \text{Min } i : \underline{0 \leq i < \#ys + 1} \wedge (y \blacktriangleright ys)!i = e : i \rangle$
= { lógica }

$\langle \text{Min } i : (i = 0 \vee 1 \leq i < \#ys + 1) \wedge (y \blacktriangleright ys)!i = e : i \rangle$

= { distributividad }

$\langle \text{Min } i : (i = 0 \wedge (y \blacktriangleright ys)!i = e) \vee (1 \leq i < \#ys + 1 \wedge (y \blacktriangleright ys)!i = e) : i \rangle$

= { partición de rango }

$\langle \text{Min } i : \underline{i = 0 \wedge (y \blacktriangleright ys)!i = e} : i \rangle \min \langle \text{Min } i : 1 \leq i < \#ys + 1 \wedge (y \blacktriangleright ys)!i = e : i \rangle$

(PARENTESIS: el 1er rango podría ser rango unitario ($i \in \{0\}$), o rango vacío ($i \in \{\}$).)

= { usamos Leibniz: reemplazo de iguales por iguales en una conjunción }

$\langle \text{Min } i : i = 0 \wedge \underline{(y \blacktriangleright ys)!0 = e} : i \rangle \min \langle \text{Min } i : 1 \leq i < \#ys + 1 \wedge (y \blacktriangleright ys)!i = e : i \rangle$

= { def. de ! }

$\langle \text{Min } i : i = 0 \wedge \underline{y = e} : i \rangle \min \langle \text{Min } i : 1 \leq i < \#ys + 1 \wedge (y \blacktriangleright ys)!i = e : i \rangle$

- Caso $y = e$: ← esto es un análisis por casos dentro de una derivación

= { sustitución }

$\langle \text{Min } i : i = 0 \wedge \text{True} : i \rangle \min \langle \text{Min } i : 1 \leq i < \#ys + 1 \wedge (y \blacktriangleright ys)!i = e : i \rangle$

= { lógica }

$\langle \text{Min } i : i = 0 : i \rangle \min \langle \text{Min } i : 1 \leq i < \#ys + 1 \wedge (y \blacktriangleright ys)!i = e : i \rangle$

= { rango unitario }

0 $\min \langle \underline{\text{Min } i : 1 \leq i < \#ys + 1 \wedge (y \blacktriangleright ys)!i = e} : i \rangle$

= { cambio de variable $i \rightarrow i + 1$ }

0 $\min \langle \text{Min } i : 1 \leq i + 1 < \#ys + 1 \wedge (y \blacktriangleright ys)!(i+1) = e : i + 1 \rangle$

= { resto 1 en la desigualdad }

0 $\min \langle \text{Min } i : 0 \leq i < \#ys \wedge (y \blacktriangleright ys)!(i+1) = e : i + 1 \rangle$

= { def. de ! }

0 $\min \langle \text{Min } i : 0 \leq i < \#ys \wedge ys!i = e : i + 1 \rangle$

= { + distribuye con Min (A6) }

0 $\min (\langle \text{Min } i : 0 \leq i < \#ys \wedge ys!i = e : i \rangle + 1)$

= { H.I. }

0 $\min (\text{busca.e.ys} + 1)$

= { 0 absorbe porque $\text{busca.e.ys} + 1 \geq 0$ siempre ya que $\text{busca.e.ys} \in \text{Nat}$ } PASO OPCIONAL

0

- Caso $y \neq e$: $\leftarrow$ importante que las guardas del análisis por casos sean “programables”
 - $= \{ \text{sustitución} \}$
 - $\langle \text{Min } i : i = 0 \wedge \text{False} : i \rangle \min \langle \text{Min } i : 1 \leq i < \#ys + 1 \wedge (y \blacktriangleright ys)!i = e : i \rangle$
 - $= \{ \text{lógica} \}$
 - $\langle \text{Min } i : \text{False} : i \rangle \min \langle \text{Min } i : 1 \leq i < \#ys + 1 \wedge (y \blacktriangleright ys)!i = e : i \rangle$
 - $= \{ \text{rango vacío} \}$
 - $+ \text{infinito} \min \langle \text{Min } i : 1 \leq i < \#ys + 1 \wedge (y \blacktriangleright ys)!i = e : i \rangle$
 - $= \{ \text{neutro del min} \}$
 - $\langle \text{Min } i : 1 \leq i < \#ys + 1 \wedge (y \blacktriangleright ys)!i = e : i \rangle$
 - $= \{ \text{mismos pasos que en el otro caso} \}$
 - busca.e.ys + 1

Resultado final: (esto es un programa válido en el teórico práctico)

$$\text{busca} : A \rightarrow [A] \rightarrow \text{Nat}$$

busca.e.[] $\doteq +\text{infinito}$

$$\text{busca.e.}(x \blacktriangleright x_s) \doteq (x = e \rightarrow 0 \quad \min \quad (\text{busca.e.ys} + 1) \\ \square x \neq e \rightarrow \text{busca.e.ys} + 1 \\)$$

Alternativamente (y mejor):

$$\text{busca.e.}(x \blacktriangleright xs) \doteq (x = e \rightarrow 0 \\ \quad \square x \neq e \rightarrow \text{busca.e.}ys + 1)$$

)

Ejercicio 5c

Especificar y derivar la función creciente : $[Int] \rightarrow Bool$ que me dice si una lista está ordenada crecientemente.

Dos formas de encararlo.

Una:

$creciente.xs = \langle \forall i : 0 \leq i < \#xs - 1 : xs ! i \leq xs ! (i + 1) \rangle$

o bien

$creciente.xs = \langle \forall i : 1 \leq i < \#xs : xs ! (i - 1) \leq xs ! i \rangle$

Otra:

$creciente.xs = \langle \forall i, j : 0 \leq i < j < \#xs : xs ! i \leq xs ! j \rangle$

Comentario: Ambas están bien (es subjetivo cuál de las dos es más natural), pero para derivar no va a convenir más una que la otra. ¿Cuál? La primera porque tiene menos términos que calcular. Si tengo $\#xs = 5$, la 1ra forma tiene 4 términos (comparaciones), y la 2da $5 * 4 / 2$ (somos vagos pero seguro son más que 4).